Combining Mechanical and Agentic Specification Inference

Wolfgang Grieskamp,* Teng Zhang, and Vineeth Kashyap
Aptos Labs, Palo Alto, USA

Abstract

We describe a specification inference tool that combines mechanical weakest-precondition (WP) analysis with an agentic coding CLI such as Claude Code. It is implemented for Move and the Move Prover, but the approach is general enough to be suitable for any Rust-style language. Inference reduces the boilerplate of verification which often hit applications in practice: to establish a high-level property such as a global state invariant, pre- and post-conditions for the supporting functions typically have to be written for the whole software system, a laborius and tedious task. A Model Context Protocol (MCP) service exposes the WP analysis and the verifier to the coding agent. The WP analysis provides a sound, mechanical baseline for inference; the AI is used precisely where WP is weakest — for loop invariants and high-level idiomatic specifications such as monotonicity, conservation, and structural invariants. The verifier serves as the oracle that decides whether the generated specs are valid, and the agent is equipped to generate proof hints and to refine the inferred specification until verification succeeds. We evaluate the approach on 16 retained targets drawn from a private perpetual-trading codebase, comparing three inference tactics — one without WP and two with — over four replicates each. At the stated GLM-5.3 prices, the WP-backed tactics cost about 31% less and leave fewer contracts incomplete, though the unaided agent still produces a complete specification in 60 of its 64 sessions.

1 Introduction

A roadblock in program verification adoption is the *cost of specification*. The properties developers want to establish are often high-level: that a call chain aborts only under stated preconditions, that an API entrypoint satisfies postconditions, that data is well-formed with respect to an invariant, or that a resource cannot be accessed without required permissions. But to verify those properties developers have to write specifications for every single function in the system.

Modern verifiers automatically abstract from imperative code using Dijkstra's weakest-precondition transformation (WP) [10]. The problem is that WP stops at loops. Most loops require *loop invariants*, which eliminate the cycle: the invariant serves as the induction hypothesis, and the loop is replaced by an acyclic fragment WP can propagate through [6]. There is no complete technique to synthesize such invariants, even though there has been a lot of work in this area, going back to Cousot and Halbwachs [8]. Loop invariants have to be provided manually in realistic applications of program verification. The invariants are not only needed for the code which is the target of verification, they are also needed for all dependencies, unless those themselves have fully specified function interfaces.

Where the human is in the loop to solve tedious problems, LLMs are there to help. Specification inference via LLMs has been investigated as early as 2023 [21, 16]. While the success rate in these early years wasn't high, LLMs have drastically improved since then. In the 2026 era of frontier models, it appears quite realistic to use a well-instructed agent to infer specifications, including loop

*Corresponding author: wg@aptoslabs.com

Preprint.

invariants, specifically since the agent receives feedback from the verifier: the program must, by definition, conform to the inferred specification, and if not, a decent verifier will produce a meaningful diagnosis.

In this work we are going one step further: we supply the agent with a WP tool which can infer precise, simplified specifications, provided loop invariants are given. The intended workflow is that the agent requests specs for a program, the tool comes back either with success, then the task is done, or with information about the missing loop invariants. The agent then asks the model to infer those invariants, and we loop back to the start.

The most interesting question with such a setup is whether the capability to run WP actually improves specification inference, and whether the overhead introduced by model/tool interaction is worth it. We investigate this question empirically by comparing pure agent and combined agent/WP on a corpus of examples drawn out of a private codebase (the one for the perpetual trading system Decibel [9]). The fact that we choose a private codebase is methodologically important, as discussed later. The currency of comparison is US dollars under the stated token prices; we use GLM-5.3 [24] as the model and the Claude Code SDK as the harness. The results show that WP-based inference, at least with today's models, beats the agent-only approach on average by 31% lower cost.

The combination of agentic and mechanical inference as discussed here has been implemented in the Move Prover (MVP). MVP has been developed as part of the Libra/Diem project at Meta, targeting the Move smart contract language, and transitioned into Aptos Labs in 2022. The Move language has many similarities to Rust: safe reference semantics, linear types, algebraic data types, higher-order functions. Those traits are what the mechanical half of our approach relies on — specifically the ability to eliminate references (Sec. 3) — so while we implement and evaluate it for Move, we expect it to carry over to verifiers for Rust and similar languages. Move has been used to specify and verify Diem's international payment system and also for critical components of the Aptos Blockchain [20, 2]. The Prover supports pre/post conditions, abort conditions, state/data/loop invariants, and higher-order functions, in first-order predicate logic. It has an efficient encoding in SMT/z3 via Boogie, which has been described in [11, 14]. For more details, see the Move book [3].

2 Inference Session Example

We illustrate the workflow of specification inference in our system by examples. The examples are written in Move, but should be understandable without knowing details about the language. For a reference, see the Move Book [3].

Consider a coding agent like Claude Code. The Move Prover is connected to the agent via an MCP service. The service offers (a) a tool for launching verification and (b) a tool for running WP-based spec inference. The agent has a set of skills which describe language, verification techniques, and the MCP tools in detail.

The example we look at is an iterative computation of the exponential, b^e:

user

```
fun pow(base: u64, exp: u64): u64 {
  let result = 1;
  let i = 0;
  while (i < exp) {
    result = result * base;
    i = i + 1;
  }
  result
}
```

Based on the instructions, the first thing the agent does is call WP. The answer points to the missing loop invariant, including unrolling the loop symbolically and showing the values of the affected loop variables at each entry:

wp tool

```
WP inferred ‘vacuous‘ conditions after this loop without an invariant. ..
  --> sources/pow.move:5:14
  |
```

```
5 |       while (i < exp) {
  |             ^
  |
  = loop-invariant evidence
  = bounded loop-head facts :
      head[0]: head[0].i = 0
               head[0].result = 1
      head[1]: 0 < exp ⟹ head[1].i = 1
               0 < exp ⟹ head[1].result = base
      head[2]: 1 < exp ⟹ head[2].i = 2
               1 < exp ⟹ head[2].result = base * base
      ..
```

Based on the diagnosis, the agent next derives the loop invariant. A recursive specification helper function is introduced for accumulating the result. Before returning the result, the agent uses the verification tool to check whether its solution verifies:

agent

```
fun pow(base: u64, exp: u64): u64 {
  ..
  while (i < exp) {
    result = result * base;
    i = i + 1;
  } spec {
    invariant [inferred] i ≤ exp;
    invariant [inferred] result = math_pow(base, i);
  }
  ..
}
// specification helper function
spec fun math_pow(base: num, exp: num): num {
  if (exp ≤ 0) { 1 } else { math_pow(base, exp - 1) * base }
}
```

In the next turn the agent runs the WP MCP tool again. With the loop invariant present, the complete specification is derived. However, the aborts condition is marked as `sathard` by the WP tool. The marker is set when a derived condition is exact but its satisfiability is expected to be hard for the solver — here because of the existential quantification, which says that the computation will abort if for some `y` the exponent multiplication overflows. Note that the Move Prover uses unbounded integers, `num`, in the specification language, making it possible to reason about overflow in the programming language's `u64`:

wp tool

```
spec pow { // post and aborts condition for pow
    ensures [inferred] result = math_pow(base, exp);
    aborts_if [inferred = sathard] ∃ y: u64:
      math_pow(base, y) * base > MAX_U64
      && (∀ x: num: 0 ≤ x && x ≤ y ⟹ math_pow(base, x) ≤ MAX_U64)
      && y < exp
}
```

The agent tries to verify this specification, but it times out with the following diagnosis:

verify tool

```
verification failed:
error: verification out of resources/timeout (timeout set to 5s)
 = timeout analysis replay: mixed quantifier and nonlinear arithmetic
 = top quantifier activity:
     quantifier at sources/pow.move:24 -- 55,793+ observed instantiations
```

The quantifier the diagnosis is pointing to is the defining recursive axiom of the specification helper function `math_pow`, which is apparently endlessly unfolded in order to prove the `sathard` aborts condition. In the next turn, the agent introduces lemmas to characterize the monotonicity of multiplication, which allows them to rewrite the aborts condition:

agent

```
spec pow {
  pragma opaque;
    ensures [inferred] result = math_pow(base, exp);
    /// The partial products are non-decreasing, so the loop
    /// overflows exactly when its last multiplication does.
    aborts_if [inferred] exp > 0 && math_pow(base, exp - 1) * base > MAX_U64;
} proof {
    // Bounds the multiplication of every iteration by the last one; the
    // trigger fires on the `math_pow(base, i)` of the loop invariant.
    ∀ k: num {math_pow(base, k)} apply math_pow_last_is_max(base, k, exp);
}
spec lemma math_pow_last_is_max(base: num, k: num, n: num) {
  requires base ≥ 0 && 0 ≤ k && k < n;
  ensures math_pow(base, k) * base ≤ math_pow(base, n - 1) * base;
} proof {
  apply math_pow_mono(base, k + 1, n);
}
spec lemma math_pow_mono ...  // left out for brevity
```

3 Weakest-Precondition Analysis

We run a textbook WP analysis [10, 6] over the bytecode MVP hands to its Boogie back-end, and read off abort conditions, modifies clauses, and consequence-style postconditions. The novelty here is not in the WP algorithm, but in the *shape of the bytecode it operates on*: by the time the analysis runs, MVP's standard transformation pipeline has already eliminated all Move references and injected every available specification as in-line `assume`/`assert` statements. The result is a reference-free, contract-annotated control flow on which backward predicate propagation reduces to the classical case.

Reference elimination. Move has Rust-style references: `&T` for shared reads and `&mut T` for exclusive writes, both with a stack-bounded lifetime. Predicate-transformer reasoning over such references is awkward, because the meaning of `*r` depends on which root location `r` was derived from. MVP avoids this by rewriting bytecode into a reference-free intermediate form [11]. Shared references are inlined: `&T` disappears and reads through it become reads of the underlying value. Mutable references are replaced by in/out parameters.

Specification instrumentation. Before WP runs, the bytecode passes through MVP's regular specification instrumentation. Each function-level `requires`, `aborts_if`, `ensures`, or `modifies` clause, and each loop `invariant`, is compiled into an `assume` or `assert` at the appropriate program point. Calls are instrumented uniformly via the *behavioral predicates* of [14]: `requires_of<f>`, `aborts_of<f>`, `ensures_of<f>`, `modifies_of<f>`, and `result_of<f>` reify a callee's contract as predicates over the call site's arguments and result. Loops are broken at their headers via the standard *cut-point* encoding of Barnett and Leino for unstructured programs [6], with the loop `invariant` playing the role of cut-point predicate: the back-edge is severed, the invariant is asserted before and after each iteration, and the loop-modified locations are havoced in between, leaving an acyclic fragment through which WP propagates as through any straight-line code.

At the input to WP, the bytecode therefore looks like a sequence of pure assignments interleaved with `assume` and `assert` over plain values; the surface-level features (references, resources, calls, loops, user specifications) are gone.

4 Tactics and Skills

The agentic side of the workflow is delivered as a plugin to a general-purpose coding agent: skills that instruct it, hooks that check generated Move as it is written, and the Move toolchain exposed as tools it can call. The setup is at [4]; we summarize the design.

Inference tactics. Inference runs in one of three *tactics*, which differ in whether the WP tool is available and who decides when to use it.

– *agent-only:* WP is unavailable. The contract and any loop invariants are derived directly from the implementation and the contracts of its dependencies, checked as the work proceeds.
– *hybrid-guided:* the workflow of Sec. 2, prescribed as a fixed order — run WP over the scope, take one warned function and write an invariant from the bounded loop-head facts, rerun WP on that function alone until the warning is gone, simplify what WP derived, then check.
– *hybrid-flexible:* WP is available as one inference pass among others, and the agent decides whether and when to call it and how to combine it with its own reasoning.

Skills. A tactic selects the *plan* — the sequence of tasks and which of them consult WP— and nothing else. Everything a tactic would otherwise have to rediscover is shared reference material, identical in all three tactics: the specification language, the rules for writing and editing specs, the verification sub-workflow, and the toolchain's capabilities and limits. Only the description of the WP tool itself is withheld where the tool is absent; the underlying idea of backward reasoning, and what a loop without an invariant costs, is background every tactic receives.

Three areas of that guidance carry most of the weight.

– *Loop invariants:* when to introduce recursive helper functions for iterative quantities, and when to prefer state-level invariants on resources — assumed by the prover at every call site, including each loop iteration — over per-loop inductive arguments.
– *Simplification:* discard vacuous conditions; replace unbounded quantifiers, a frequent source of SMT timeouts, with bounded or closed-form equivalents; and require trigger annotations on the surviving ones.
– *Timeout resolution:* a hierarchy of escalations — strengthen state-level invariants, then introduce spec helpers, then lemmas, then explicit proof scripts — with a strict prohibition on weakening abort or postconditions to silence a timeout, since this trades a real property for the appearance of progress.

Independently of the tactic, every emitted condition is tagged with an `[inferred]` marker, so that re-runs are idempotent and never disturb user-written specifications.

5 Evaluation

Setup. The evaluation setup aims at testing the three different tactics (arms) mentioned above – *agent-only*, *hybrid-flexible*, and *hybrid-guided*, which are run under the same conditions, using Claude Code with GLM-5.3 as the underlying model.

The corpus is drawn from Etna, the private Move code of the perpetual trading system Decibel, plus five targets that are not from it: three functions from a public Move library, and two written for the corpus. None of the five carries an upstream specification either. A private codebase matters here: the model cannot have seen these functions, or any specification of them, during training, so a contract has to be inferred rather than recalled.

During selection, every candidate target is classified by the features (strata) its contract has to account for — loops and accumulators, early returns and breaks, arithmetic overflow and underflow, casts and truncation, mutable references and in-place permutation, function values, quantifiers, composition through a callee's contract, and others, 34 such features in all. A target is admitted only if it proves in a few seconds against a hand-written reference specification. The result is **17 tasks** covering all 34 features across 12 modules, 9 of which contain loops. With 3 arms and 4 replicates this initially made 204 cells. After inspecting the run traces, we exclude `VS-shares-002` in all three arms and all four replicates because hybrid-flexible replicate 4 encountered WP filter-selection failures and generated specifications outside the target scope. All results below use the remaining **16 tasks and 192 cells**. This exclusion was made after observing the results; the original 17-task comparison gave about 28% lower mean cost for both hybrids.

Scoring. A cell reaches *operational success* when its specification compiles, verifies, and passes a set of tests which check its *completeness*. Completeness is a significant problem for agent-based inference, since postconditions in particular can be weaker than the actual behavior of the function, and agents tend to select the shortest path to a solution. The skills make it very clear that completeness is a requirement, but an agent may ignore this. There is no deductive method for checking completeness

Table 1: Cost in US dollars per cell by target: mean over the four replicates, mean input/output tokens in thousands (I/O k), the hybrid mean cost as a percentage of agent-only cost (vs. agent), and whether the specification killed every mutant (✓) or was defeated by one (✗, subscripted with the count where more than one replicate failed). ○ marks a task-arm whose replicate was lost to an infrastructure fault. Summed over the retained tasks the arms cost \$24.91, \$17.16 and \$17.31, i.e. 0.69× and 0.69× agent-only. Input counts include uncached, cache-read, and cache-creation tokens.

	agent-only			hybrid-guided				hybrid-flexible			
target	cost	I/O k	ok	cost	vs. agent	I/O k	ok	cost	vs. agent	I/O k	ok
`BA-base-012`	\$0.380	710/31.3	✓	\$0.251	66%	476/19.4	✓	\$0.311	82%	529/27.6	✓
`LP-price-021`	\$0.197	475/9.0	✓	\$0.228	116%	514/12.7	✓	\$0.207	105%	465/11.2	✓
`MM-min-013`	\$0.126	284/6.3	✓	\$0.141	112%	336/6.2	✓	\$0.134	106%	295/7.0	✓
`OV-order-006`	\$0.097	210/5.0	✓	\$0.076	78%	165/3.3	✓	\$0.080	83%	180/3.3	✓
`QP-part-025`	\$0.698	1593/46.0	$✗_2$	\$0.490	70%	1008/35.6	✓	\$0.448	64%	877/35.2	✗
`SM-select-022`	\$0.406	879/27.9	✓	\$0.434	107%	833/34.7	✓	\$0.276	68%	547/20.0	✓
`TL-lev-020`	\$0.205	490/10.3	✓	\$0.313	153%	628/22.6	✓	\$0.206	100%	414/13.5	✓
`TR-cancel-026`	\$0.442	1010/27.0	✓	\$0.335	76%	722/21.6	✓	\$0.359	81%	833/21.1	✓
`TR-discard-011`	\$0.238	561/12.4	✓	\$0.217	91%	491/12.1	✓	\$0.292	122%	668/16.7	✓
`TR-order-010`	\$0.287	650/17.2	✓	\$0.197	69%	428/11.6	✓	\$0.245	85%	520/15.3	✓
`TS-trial-019`	\$0.407	821/31.1	✓	\$0.371	91%	710/26.0	○	\$0.317	78%	642/22.8	✓
`UC-credits-008`	\$0.806	2322/31.0	✗	\$0.215	27%	495/11.6	✓	\$0.389	48%	1033/16.9	✓
`VS-contrib-003`	\$1.028	2800/45.9	✗	\$0.182	18%	400/9.9	✓	\$0.155	15%	317/9.3	✓
`VS-fees-001`	\$0.233	479/14.9	✓	\$0.208	90%	430/13.0	✓	\$0.200	86%	416/11.9	✓
`VS-redeem-004`	\$0.555	1290/33.5	✓	\$0.516	93%	1167/32.1	✓	\$0.603	109%	1340/38.2	○
`WU-consume-023`	\$0.121	240/7.6	✓	\$0.116	96%	236/6.7	✓	\$0.106	87%	198/6.7	✓
all 16	\$0.389	926/22.3	60/64	\$0.268	69%	565/17.4	63/63	\$0.270	69%	580/17.3	62/63

in the general case. WP guarantees completeness only in the presence of complete loop invariants and complete specs of callees; if completeness is not reached, it may be able to generate a warning. The agent-only tactic does not have this mechanism.

The harness evaluates completeness using *mutation testing* with a set of hand-generated mutants, changing the verified implementation. To pass the test, each of the mutants needs to be killed — that is, verification fails when the mutant is verified against the inferred spec.

Cost. Costs in US dollars use the GLM-5.3 prices recorded for the round: \$1.40 per million uncached input tokens, \$0.26 per million cached input tokens, and \$4.40 per million output tokens. For the corresponding token counts I_u, I_c, and O, the cost is $cost_{\mathrm{USD}} = (1.40I_u + 0.26I_c + 4.40O)/10^6$. Cache-creation tokens are currently uncharged and contribute zero; we recompute from the recorded token counters rather than use the SDK's cost estimate. Table 1 gives cost and input/output tokens per cell, by target. Both hybrid arms are about 31% cheaper in the observed mean; the 12 agent-only cells above \$0.50 become six and five. The saving is uneven. The hybrids are cheaper on 12 and 11 of the 16 targets, and the round-level figure is carried by a few of them rather than spread evenly: `VS-contrib-003` costs agent-only 5.6× what it costs hybrid-guided and 6.6× what it costs hybrid-flexible, while `TL-lev-020` costs 1.5× as much under hybrid-guided as without WP. We quantify precision with 95% percentile bootstrap confidence intervals. For the aggregate estimates, we hold the 16 tasks fixed and resample four replicate blocks with replacement within each task, keeping all three arms of a block together (100,000 draws, seed 20260907). The hybrid/agent-only mean-cost ratios are 0.689 [0.559, 0.855] for guided and 0.695 [0.538, 0.901] for flexible. Both intervals lie below 1, supporting lower expected cost on this fixed corpus; they do not quantify generalization to other tasks.

The mechanism is visible in the tool counts. The agent-only arm performs about 80% more Edit calls (362 versus 201) and three times as many verify calls (127 versus 39) as hybrid-flexible. The self-reports describe the agent-only arm deriving contracts through counterexamples and self-designed mutation probes, while hybrid answers also describe using WP output to identify missing loop invariants. The arms produce similar kinds of specifications — opaque contracts, recursive spec helpers, and loop invariants — but differ in how they arrive at them.

Failure points. Five of the 192 retained cells failed the mutation test, and 2 more were lost to infrastructure faults; Table 1 marks where. Every failure is an agent-only cell but one. Three are `weak_contract`: a mutant survived, in every case leaving `normal-result` unconstrained. Two of

those are `QP-part-025`, the in-place permutation, where the surviving class was "right partition point, wrong contents" — a contract that pins where the partition lands but not what it contains. The fourth agent-only failure is different in kind: on `UC-credits-008` the arm changed the implementation rather than specify it, which the edit policy forbids. The one hybrid failure is on the same target and of the same class as two of the agent-only ones. The two infrastructure faults are unrelated to inference (the MCP supervisor restarted the session) and fall one in each hybrid arm.

Pooling the two hybrid arms, agent-only left a mutant alive in 4 of its 64 cells against 1 of the hybrids' 126 — 6.25% against 0.79% — and a Fisher exact test gives $p = 0.045$, two-sided. The comparison rests on five events, so it is fragile: the test says the difference is unlikely to be chance, not how large it is, and a single cell moving either way would change that.

6 Discussion and Conclusion

6.1 Interpreting the Results

The headline of Sec. 5 is that both WP-backed tactics cost about 31% less than agent-only while producing mutation-complete specifications in 63/63 hybrid-guided and 62/63 hybrid-flexible cells, compared with 60/64 agent-only cells; the two cells lost to infrastructure are excluded from these fractions. The more informative result is how narrow the margin is.

Agent-only does surprisingly well. Given no WP tool at all, the direct tactic produced a verifying, mutation-complete specification in 60 of its 64 sessions. It found the same recursive spec helpers and loop invariants as the hybrids, at nearly the same rates, and it got there by a recognizable method: it wrote its own mutation probes and let counterexamples drive the repair. The WP tool reduces dollar cost by about 31% and avoids four failures — real, but not the difference between working and not working.

Two things temper that. The targets are individual functions of a few dozen lines, and while several need genuine invention — an invented lemma, a permutation contract, a bound that holds only through a callee's contract — none requires reasoning across a module or a call graph of any depth. And the four failures are not spread evenly: they concentrate where the contract has to pin down a whole data structure rather than a scalar result, which is where a mechanical baseline has the most to contribute. We expect the gap to widen with the size of the target, and we have no data on that yet.

The next model generation. The round used a single model, and the obvious question is whether a stronger one closes the gap. It plausibly narrows it: most of what WP supplies here is a diagnosis — *this loop has no invariant, and here are its first iterations* — and a model that finds the invariant unaided needs the diagnosis less. The agent-only sessions that cost the most were the ones searching without one.

We do not read that as WP becoming redundant. What WP derives is exact and free of tokens, and the hybrid arms also spend their budget elsewhere: on simplifying a machine-shaped contract into a readable one, which is model work that gets better with the model. A stronger model should improve both arms, and the interesting measurement is not which wins but whether the mechanical component keeps removing the same class of failure — the incomplete abort condition, the vacuous postcondition after a havocked loop — that no amount of fluency prevents.

Open questions. The evaluation leaves the questions of scale open. Its targets are functions, not modules; its corpus is one codebase in one domain; and its cost measure is dollars under one model's price schedule. We plan to apply inference retrospectively to the Aptos Framework [2], which already carries hand-written specifications, and to compare the inferred contracts against them — a comparison this round cannot make, since its targets deliberately have no specification of their own.

6.2 Related Work

Specification inference predates large language models — Daikon [12] learned likely invariants from program traces — but the techniques that followed stayed tied to particular property classes and source languages, and the advent of LLMs has revived interest in the area. Pei et al. [21] showed that fine-tuned LLMs can predict program invariants statically at quality competitive with dynamic analysis. Kamath et al. [16] pair an LLM that *proposes* loop invariants with a symbolic checker

that *validates* them, retrying on failure — a propose-and-validate loop that recurs in much of the subsequent work. For Verus, AlphaVerus [1] bootstraps verified code by self-improving translation with tree search over verifier feedback, and VeriStruct [22] synthesizes Verus-style specifications and proofs for Rust functions, modules, and data structures through planner-orchestrated phases. For smart contracts in Solidity, PropertyGPT [18] retrieves human-written Certora properties as input examples for specification generation. Most directly related to our work, MSG [13] targets the same Move specification language through a skill-based agentic design that consumes verifier feedback beyond a binary success/failure verdict.

Across these systems the LLM remains responsible for the entire specification; the verifier serves as a binary oracle or, in MSG and our setup, as a feedback channel, but never as a contributor of mechanically derived content.

A separate line of work, exemplified by DafnyPro [5] and AutoVerus [23], tackles the complementary problem of synthesizing the auxiliary proof annotations a verifier needs when the specification is already given. Our plugin can also synthesize proof hints (Sec. 2), but this is not the focus of this paper.

Reference elimination, the bytecode transformation underlying our WP analysis (Sec. 3), was introduced in [11], and equivalents exist for Rust. Aeneas [15] compiles Rust to a pure functional model in a target prover (Lean, F*, Coq, HOL4) via forward and backward functions, while RustHorn [19] encodes a mutable borrow as a pair of its current value and a *prophecy* for its value at the end of the borrow, reducing verification to constrained Horn clauses. Neither infers specifications — in Aeneas the user writes them in the target prover, against the translated model — and both consume the reference-free form only in the forward direction. Our use of the transformation is the opposite: WP runs on the reference-free form, but the conditions it derives have to be carried *back* through the elimination, so that what the user reads and edits is stated over the original references rather than over in/out parameters or prophecy variables. To our knowledge this inversion has not been described before; it is what makes a reference-eliminating pipeline usable for inference rather than only for proving. It is straightforward for *path-based* eliminations such as ours and Aeneas's, where an eliminated reference still corresponds to a location the source can name. Whether it carries to a prophecy encoding, where the borrow's final value has no source-level counterpart, we do not know.

Our backend IVL, Boogie [7, 17], also performs WP on the procedural code passed in [6] and passes the result on to the SMT solver. One major difference from our approach is that we aim to generate user-readable conditions, in terms of Move, whereas Boogie's WP is not intended for external consumption. This is reflected in our mapping of references back to the source language, and in the significant effort put into expression simplification, since raw WP predicates can be very verbose.

In summary, two aspects distinguish our setup from the work above. First, to our knowledge, no prior LLM-based spec-inference system uses a sound mechanical baseline alongside the LLM. WP itself produces a substantial part of the inferred specification — abort conditions, modifies clauses, and consequence-style postconditions — so the AI is left only with what WP cannot recover (loop invariants, high-level conservation laws), and the two streams are validated together by the Move Prover. The AI side runs a propose-observe-refine loop like the systems above; the mechanical baseline narrows the LLM's search space.

Second, the workflow is delivered as a plugin to a general-purpose, interactive coding agent (Claude Code) via skills, edit-time hooks, and an MCP server, rather than as a custom-built tool: the agent participates in the loop and – most importantly – the user can intervene at any point. To our knowledge, no prior LLM-based inference system for verification languages has this kind of coding-agent integration, including agentic ones such as MSG.

6.3 Conclusion

We have presented a specification inference tool for the Move Prover that combines a sound, mechanical WP analysis over Move bytecode with an agentic coding CLI; the prover serves as the oracle for both signals. The two inference streams compose in a single instrumentation pipeline, and the workflow is delivered as a Claude Code plugin — skills, hooks, and an MCP server — so inference runs inside the same conversational loop the user already writes Move in. A controlled comparison on 16 retained targets drawn from a private perpetual-trading codebase — three tactics, four replicates

each — found the WP-backed tactics about 31% cheaper at the stated GLM-5.3 prices and less prone to leaving a contract incomplete, while the unaided agent still produced a complete specification in 60 of its 64 sessions. Whether that margin widens with the size of the target, and whether it survives a stronger model, are the questions we take up next. Neither the WP construction nor the agentic workflow is specific to Move: both rest on reference elimination and on a verifier that consumes contracts, so we expect the approach to transfer to Rust-like languages and their verifiers.

References

[1] Pranjal Aggarwal, Bryan Parno, and Sean Welleck. AlphaVerus: Bootstrapping formally verified code generation through self-improving translation and treefinement. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, volume 267 of *Proceedings of Machine Learning Research*, 2025.

[2] Aptos Labs. The Aptos Framework Book. https://aptos-labs.github.io/framework-book/, 2023. Accessed: 2026-05-09.

[3] Aptos Labs. The Move on Aptos Book. https://aptos-labs.github.io/move-book/, 2023. Accessed: 2026-05-09.

[4] Aptos Labs. MoveFlow: AI-assisted Move smart contract development. https://github.com/aptos-labs/aptos-core/tree/d8be469f7f/aptos-move/flow, 2026. Commit `d8be469f7f`, accessed 2026-05-11.

[5] Debangshu Banerjee, Olivier Bouissou, and Stefan Zetzsche. DafnyPro: LLM-assisted automated verification for Dafny programs, 2026. URL https://arxiv.org/abs/2601.05385. Dafny Workshop, POPL 2026.

[6] Mike Barnett and K. Rustan M. Leino. Weakest-precondition of unstructured programs. In *Proceedings of the 6th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering*, pages 82—87, New York, NY, USA, 2005. Association for Computing Machinery. ISBN 1595932399. doi: 10.1145/1108792.1108813.

[7] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K Rustan M Leino. Boogie: A modular reusable verifier for object-oriented programs. In *International Symposium on Formal Methods for Components and Objects*, pages 364–387. Springer, 2005.

[8] Patrick Cousot and Nicolas Halbwachs. Automatic discovery of linear restraints among variables of a program. In *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '78)*, pages 84–96. ACM, 1978. doi: 10.1145/512760.512770.

[9] Decibel. Decibel: A Fully Onchain Perpetuals Exchange on Aptos. https://docs.decibel.trade/, 2026. Accessed: 2026-09-04.

[10] Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975. doi: 10.1145/360933.360975.

[11] David L. Dill, Wolfgang Grieskamp, Junkil Park, Shaz Qadeer, Meng Xu, and Jingyi Emma Zhong. Fast and reliable formal verification of smart contracts with the move prover. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022)*, volume 13243 of *Lecture Notes in Computer Science*, pages 183–200. Springer, 2022. doi: 10.1007/978-3-030-99524-9_10.

[12] Michael D. Ernst, Jake Cockrell, William G. Griswold, and David Notkin. Dynamically discovering likely program invariants to support program evolution. *IEEE Transactions on Software Engineering*, 27(2):99–123, 2001. doi: 10.1109/32.908957.

[13] Yu-Fu Fu, Meng Xu, and Taesoo Kim. Agentic specification generator for Move programs. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025)*, pages 1286–1298, 2025. doi: 10.1109/ASE63991.2025.00110.

[14] Wolfgang Grieskamp, Teng Zhang, Vineeth Kashyap, and Jake Silverman. Formal verification of imperative first-class functions in Move. *CoRR*, abs/2605.10007, 2026. URL https://arxiv.org/abs/2605.10007. Preprint; to appear in FMCAD 2026.

[15] Son Ho and Jonathan Protzenko. Aeneas: Rust verification by functional translation. *Proc. ACM Program. Lang.*, 6(ICFP):116:1–116:31, 2022. doi: 10.1145/3547647.

[16] Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models, 2023. URL https://arxiv.org/abs/2311.07948.

[17] K. Rustan M. Leino and Philipp Rümmer. A polymorphic intermediate verification language: Design and logical encoding. In Javier Esparza and Rupak Majumdar, editors, *TACAS*, pages 312–327, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg. ISBN 978-3-642-12002-2.

[18] Ye Liu, Yue Xue, Daoyuan Wu, Yuqiang Sun, Yi Li, Miaolei Shi, and Yang Liu. PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation. In *Network and Distributed System Security Symposium (NDSS 2025)*, 2025. doi: 10.14722/ndss.2025.241357.

[19] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. RustHorn: CHC-based verification for Rust programs. *ACM Trans. Program. Lang. Syst.*, 43(4):1–54, 2021. doi: 10.1145/3462205.

[20] Junkil Park, Teng Zhang, Wolfgang Grieskamp, Meng Xu, Gerardo Di Giacomo, Kundu Chen, Yi Lu, and Robert Chen. Securing Aptos Framework with Formal Verification. In *5th International Workshop on Formal Methods for Blockchains (FMBC 2024)*, volume 118 of *Open Access Series in Informatics (OASIcs)*, pages 9:1–9:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. doi: 10.4230/OASIcs.FMBC.2024.9.

[21] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. Can large language models reason about program invariants? In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, volume 202 of *Proceedings of Machine Learning Research*, pages 27496–27520. PMLR, 2023. URL https://proceedings.mlr.press/v202/pei23a.html.

[22] Chuyue Sun, Yican Sun, Daneshvar Amrollahi, Ethan Zhang, Shuvendu K. Lahiri, Shan Lu, David L. Dill, and Clark Barrett. VeriStruct: AI-assisted automated verification of data-structure modules in Verus. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, Lecture Notes in Computer Science. Springer, 2026. URL https://arxiv.org/abs/2510.25015. arXiv:2510.25015.

[23] Chenyuan Yang, Xuheng Li, Md Rakib Hossain Misu, Jianan Yao, Weidong Cui, Yeyun Gong, Chris Hawblitzel, Shuvendu K. Lahiri, Jacob R. Lorch, Shuai Lu, Fan Yang, Ziqiao Zhou, and Shan Lu. AutoVerus: Automated proof generation for Rust code. *Proc. ACM Program. Lang.*, 9(OOPSLA2), 2025. doi: 10.1145/3763174.

[24] Z.ai. GLM-5.3. https://huggingface.co/zai-org/GLM-5.3, 2026. Model card. Accessed: 2026-09-04.